

SteakHouse Finance - Audit

Security Assessment

CertiK Assessed on May 8th, 2026





CertiK Assessed on May 8th, 2026

SteakHouse Finance - Audit

The security assessment was prepared by CertiK.

Executive Summary

TYPES

Bonding Curve

ECOSYSTEM

EVM Compatible

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 10/02/2025

Final report published on 05/08/2026

Vulnerability Summary

**25**
Total Findings**23**
Resolved**1**
Multi-Sig**0**
Partially Resolved**1**
Acknowledged**0**
Declined**3 Centralization**

1 Removed, 1 Multi-Sig, 1 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

2 Critical

2 Resolved



Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

1 Major

1 Resolved



Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

7 Medium

7 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

11 Minor

11 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

1 Informational

1 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | STEAKHOUSE FINANCE - AUDIT

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

Findings

[SFA-02 : Duplicate Buyers Entry Via `transferCurveBalance` Allows Users To Receive More Airdrop Tokens When Self Transfer](#)

[SFA-05 : Missing Access Control In Creator Contracts Allows Creation Fee And Cap Bounds Bypass](#)

[SFA-03 : Owner Can Withdraw Bonded ETH Via `emergencyWithdraw`](#)

[SFA-21 : Centralization Risks](#)

[SFA-28 : Centralization Risks](#)

[SFA-04 : Sell Transaction Amount Incorrectly Capped By Uncirculated Supply](#)

[SFA-06 : HEADERLESS_FEE Payment At Creation Time Can Be Bypassed](#)

[SFA-07 : Improper Conditioning Of `finalTaxRate` Validation Allows Tax Limit Bypass](#)

[SFA-08 : Unbounded Loop In `airdropBuyers` And Missing Stipend Funding](#)

[SFA-19 : Exceeding `maxEthAtCap` May Revert](#)

[SFA-22 : Lack Of TokenId Uniqueness Checks In `KitchenCreatorBasicAdvanced`](#)

[SFA-23 : StealthFee Is Not Paid In `createSuperSimpleToken` And `createZeroSimpleToken`](#)

[SFA-26 : Anti-Whale Limits Can Be Bypassed Due To Time Base Mismatch](#)

[SFA-01 : Liquidity Pool Price Manipulation During Token Graduation](#)

[SFA-09 : Unchecked ERC-20 `transfer\(\)` / `transferFrom\(\)` Call](#)

[SFA-10 : Unused Return Value](#)

[SFA-11 : Unchecked Unlock Time](#)

[SFA-12 : Payable Creation Functions Can Unintentionally Lock Native Tokens](#)

[SFA-13 : Gas-DoS On Sells Due To O\(N\) Linear Scan In `removeBuyer`](#)

[SFA-14 : Unnecessary Access In `deployToken`](#)

[SFA-18 : Potential Sandwich Attacks In Bonding Curve](#)

[SFA-24 : Possible Address Collision In `KitchenCreatorBasicAdvanced` And `KitchenCreatorSimple`](#)

[SFA-25 : Lack Of Input Validation](#)

SFA-27 : Cross-Function Reentrancy Between `buyToken` And `sellToken`

SFA-15 : Divide Before Multiply

I Optimizations

SFA-16 : Unnecessary utils contract dependency

SFA-17 : Duplicate functions `setHeaderlessPreference` and `setRemoveHeaderFlag`

I Appendix

I Disclaimer

CODEBASE | STEAKHOUSE FINANCE - AUDIT

Repository

https://github.com/steaktech/Smart_Contracts_Kitchen/tree/21296ba746aac3981e758723c37280c221f55260

Commit

[21296ba746aac3981e758723c37280c221f55260](#)

[42797cde83a940d47464549887d9fe6b2845d14b](#)

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | STEAKHOUSE FINANCE - AUDIT

This audit was conducted for SteakHouse Finance to evaluate the security and correctness of the smart contracts associated with the SteakHouse Finance - Audit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

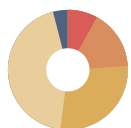
- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | STEAKHOUSE FINANCE - AUDIT

Audit scope excludes both the direct launch functionality and the token implementation; those components were out of scope for this engagement and were therefore not reviewed or tested.

In addition, the last commit the audit fully reviewed is 42797cde83a940d47464549887d9fe6b2845d14b. There has been major updates since that commit, the audit team has only checked remediation for the findings in the later commit. Any new added code and refactored code since the 42797cde83a940d47464549887d9fe6b2845d14b commit is out of the scope of the audit.

FINDINGS | STEAKHOUSE FINANCE - AUDIT



25

Total Findings

2

Critical

3

Centralization

1

Major

7

Medium

11

Minor

1

Informational

This report has been prepared for SteakHouse Finance to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 25 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
SFA-02	Duplicate Buyers Entry Via <code>transferCurveBalance</code> Allows Users To Receive More Airdrop Tokens When Self Transfer	Logical Issue	Critical	Resolved
SFA-05	Missing Access Control In Creator Contracts Allows Creation Fee And Cap Bounds Bypass	Access Control	Critical	Resolved
SFA-03	Owner Can Withdraw Bonded ETH Via <code>emergencyWithdraw</code>	Centralization	Centralization	Removed
SFA-21	Centralization Risks	Centralization	Centralization	2/3 Multi-Sig
SFA-28	Centralization Risks	Centralization	Centralization	Acknowledged
SFA-04	Sell Transaction Amount Incorrectly Capped By Uncirculated Supply	Logical Issue	Major	Resolved
SFA-06	HEADERLESS_FEE Payment At Creation Time Can Be Bypassed	Logical Issue	Medium	Resolved
SFA-07	Improper Conditioning Of <code>finalTaxRate</code> Validation Allows Tax Limit Bypass	Inconsistency	Medium	Resolved
SFA-08	Unbounded Loop In <code>_airdropBuyers</code> And Missing Stipend Funding	Logical Issue	Medium	Resolved
SFA-19	Exceeding <code>maxEthAtCap</code> May Revert	Design Issue	Medium	Resolved

ID	Title	Category	Severity	Status
SFA-22	Lack Of TokenId Uniqueness Checks In <code>KitchenCreatorBasicAdvanced</code>	Logical Issue	Medium	Resolved
SFA-23	StealthFee Is Not Paid In <code>createSuperSimpleToken</code> And <code>createZeroSimpleToken</code>	Logical Issue	Medium	Resolved
SFA-26	Anti-Whale Limits Can Be Bypassed Due To Time Base Mismatch	Logical Issue	Medium	Resolved
SFA-01	Liquidity Pool Price Manipulation During Token Graduation	Design Issue	Minor	Resolved
SFA-09	Unchecked ERC-20 <code>transfer()</code> / <code>transferFrom()</code> Call	Volatile Code	Minor	Resolved
SFA-10	Unused Return Value	Volatile Code	Minor	Resolved
SFA-11	Unchecked Unlock Time	Logical Issue	Minor	Resolved
SFA-12	Payable Creation Functions Can Unintentionally Lock Native Tokens	Logical Issue	Minor	Resolved
SFA-13	Gas-DoS On Sells Due To O(N) Linear Scan In <code>removeBuyer</code>	Logical Issue	Minor	Resolved
SFA-14	Unnecessary Access In <code>deployToken</code>	Access Control	Minor	Resolved
SFA-18	Potential Sandwich Attacks In Bonding Curve	Logical Issue	Minor	Resolved
SFA-24	Possible Address Collision In <code>KitchenCreatorBasicAdvanced</code> And <code>KitchenCreatorSimple</code>	Denial of Service	Minor	Resolved
SFA-25	Lack Of Input Validation	Volatile Code	Minor	Resolved
SFA-27	Cross-Function Reentrancy Between <code>buyToken</code> And <code>sellToken</code>	Inconsistency	Minor	Resolved

ID	Title	Category	Severity	Status
SFA-15	Divide Before Multiply	Incorrect Calculation	Informational	Resolved

SFA-02 Duplicate Buyers Entry Via `transferCurveBalance` Allows Users To Receive More Airdrop Tokens When Self Transfer

Category	Severity	Location	Status
Logical Issue	● Critical	contracts/Kitchen.sol (base): 354~376	● Resolved

Description

A logical flaw in the `transferCurveBalance` function allows a user to add duplicate entries of their own address to a token's `buyers` list by transferring their entire balance to themselves.

The vulnerability stems from the order of operations when the sender and receiver are the same address.

```
uint256 senderBal = storageContract.getUserBalance(msg.sender, token);
require(senderBal >= amount && amount > 0, "Insufficient balance");

// Update balances
storageContract.updateUserBalance(
    msg.sender,
    token,
    senderBal - amount
);

uint256 receiverBal = storageContract.getUserBalance(to, token);
if (receiverBal == 0) {
    storageContract.addBuyer(token, to);
}
storageContract.updateUserBalance(to, token, receiverBal + amount);

// Remove sender if balance zero
if (senderBal - amount == 0) {
    storageContract.removeBuyer(token, msg.sender);
}
```

When `senderBal == amount`, the function first debits the sender's balance to zero, then reads this temporary zero balance as the receiver's pre-transfer balance. This incorrect state reading triggers the `addBuyer` logic, adding a duplicate entry.

Although the user's balance is restored by the end of the transaction, the corresponding `removeBuyer` logic fails because it checks the final, non-zero balance, leaving the `buyers` list in a corrupted state with duplicate entries.

Since the buy's list is used for users to receive airdrop tokens after graduation, this allows users to inflate their balance before airdrop and receive more tokens than he supposed to.

Recommendation

We understand that the project intends to implement for functionality to give user a better experience, but this comes with the cost of complicating the system and introduce more attack vectors. We recommend remove the functionality of transfer virtual token before graduation if possible.

■ Alleviation

[SteakHouse Finance, 10/23/2025]: The team heeded the advice and resolved the issue by removing the function `transferCurveBalance` in commit [26cb07f4d12014b737a181d9a00a45f5c9b2163c](#)

SFA-05 | Missing Access Control In Creator Contracts Allows Creation Fee And Cap Bounds Bypass

Category	Severity	Location	Status
Access Control	Critical	contracts/KitchenCreatorBasicAdvanced.sol (base): 112~285; contracts/KitchenCreatorSimple.sol (base): 82~259; contracts/KitchenFactory.sol (base)	Resolved

Description

The Kitchen protocol routes token creation through factory contracts, which enforce important security and economic rules:

`_enforceCapBounds` : ensures graduation caps and total supply stay within safe limits.

`_collectFees` : ensures the protocol collects the required creation fee(including `BASE_FEE_BASIC`, `HEADERLESS_FEE` and `STEALTH_FEE`)

However, the underlying creator contracts (`KitchenCreatorBasicAdvanced`, `KitchenCreatorSimple`) expose their token creation functions publicly:

- `createBasicToken()`
- `createBasicTokenStealth()`
- `createAdvancedToken()`
- `createAdvancedTokenStealth()`
- `createSuperSimpleTokenPublic()`
- `createSuperSimpleTokenStealth()`
- `createZeroSimpleTokenPublic()`
- `createZeroSimpleTokenStealth()`

These functions contain no access control modifiers (such as `onlyFactory`), so anyone can directly call them.

As a result:

- The creator can bypass `_enforceCapBound` , deploying tokens with arbitrary supply and graduation cap.
- The creator can bypass `_collectFees` , avoiding protocol creation fees altogether.
- `_newVirtualTokenId` increments a global nonce and deterministically derives the token ID as `address(uint160(uint256(keccak256(abi.encodePacked(creator, _nonce)))))`. The only entropy used is the caller-supplied `creator` and an internal counter. Because `creator` is not bound to the actual transaction sender, any external caller of the public creation endpoints that use this helper can mint a token that is attributed to any arbitrary address. The resulting `TokenCreated` event and stored metadata will show the chosen `creator` (the victim), while the attacker controls all other

parameters (e.g., tax wallet for advanced tokens). This is a pure logic flaw in the ID derivation: it trusts an unverified parameter to define the identity namespace for token IDs.

This issue also exists in the factory contract, which does not restricts functions being called from Kitchen.

■ Recommendation

Restrict the creator functions so they can only be called from the factory.

■ Alleviation

[SteakHouse Finance, 10/24/2025]: The team heeded the advice and resolved the issue in commit [ba4e6700adee56d95dd9ed59b3587ab878cbf1ed](#)

SFA-03 | Owner Can Withdraw Bonded ETH Via `emergencyWithdraw`

Category	Severity	Location	Status
Centralization	● Centralization	<code>contracts/KitchenBondingCurve.sol</code> (base): 369~375; <code>contracts/SteakLockers.sol</code> (base): 153~159	● Removed

Description

The `KitchenBondingCurve` contract holds ETH that backs sells on the bonding curve. The `onlyOwner` function `emergencyWithdraw` allows the owner to transfer arbitrary amounts of ETH from the contract to any address at any time. If the owner (or a compromised owner key) drains ETH, sellers may be unable to redeem tokens for ETH, causing losses to users. There are no safeguards such as caps, timelocks, or withdrawal windows.

The `SteakLockers` contract holds the `lpToken` locked in the contract. The `onlyOwner` function `emergencyWithdraw` allows the owner to transfer arbitrary amounts of `lpToken` from the contract to any address at any time. If the owner (or a compromised owner key) drains the `lpToken`, as a result, users' deposited liquidity can be stolen in a rug pull scenario. There are no safeguards such as caps, timelocks, or withdrawal windows.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised; AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND

- Introduction of a DAO/governance/voting module to increase transparency and user involvement. AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles. OR
- Remove the risky functionality.

I Alleviation

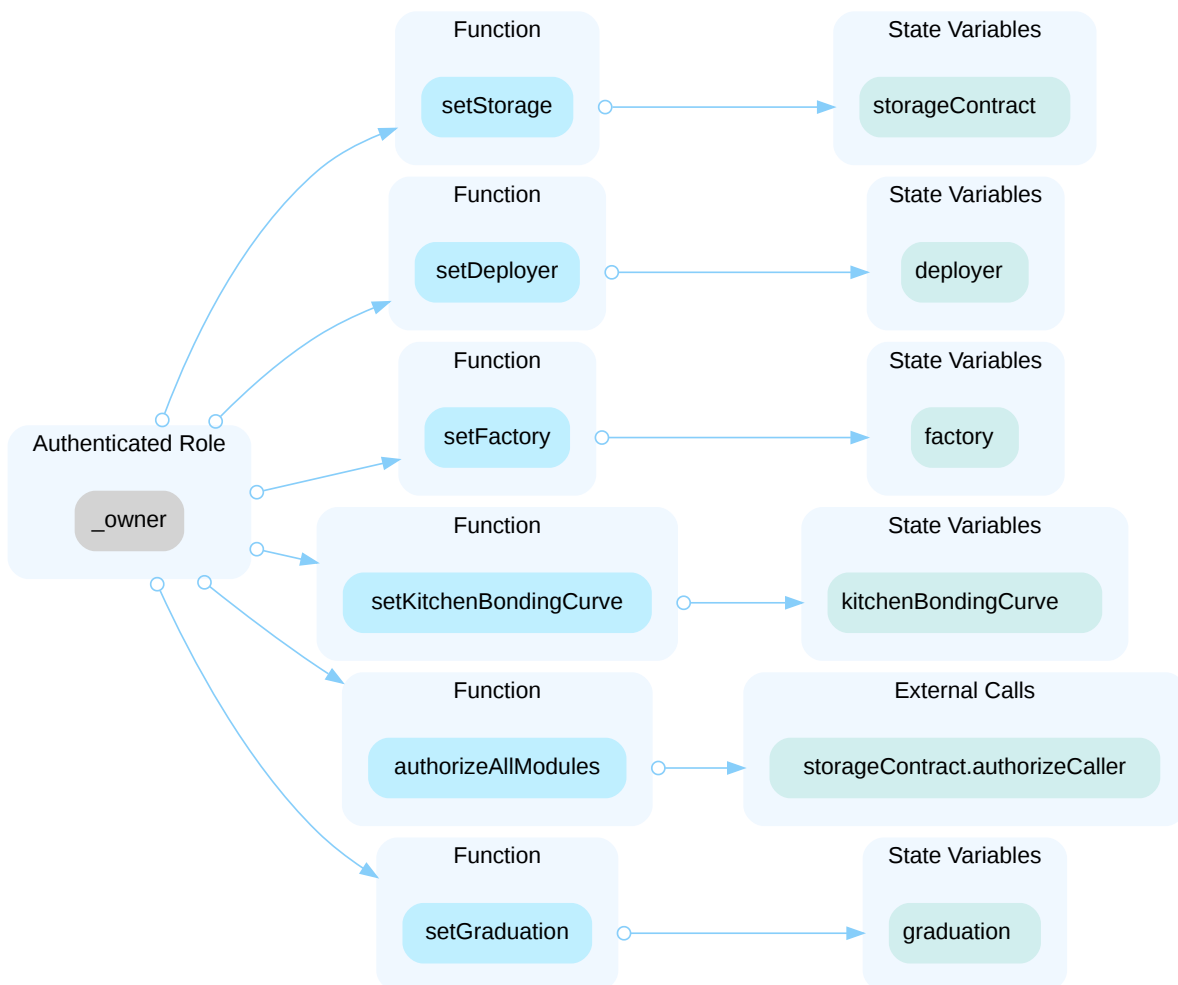
[SteakHouse Finance, 10/22/2025]: The team heeded the advice and resolved the issue by removing the `emergencyWithdraw` functions in commit [0ae00c99015b22cb2970b14be104bcf6cd84f247](#).

SFA-21 | Centralization Risks

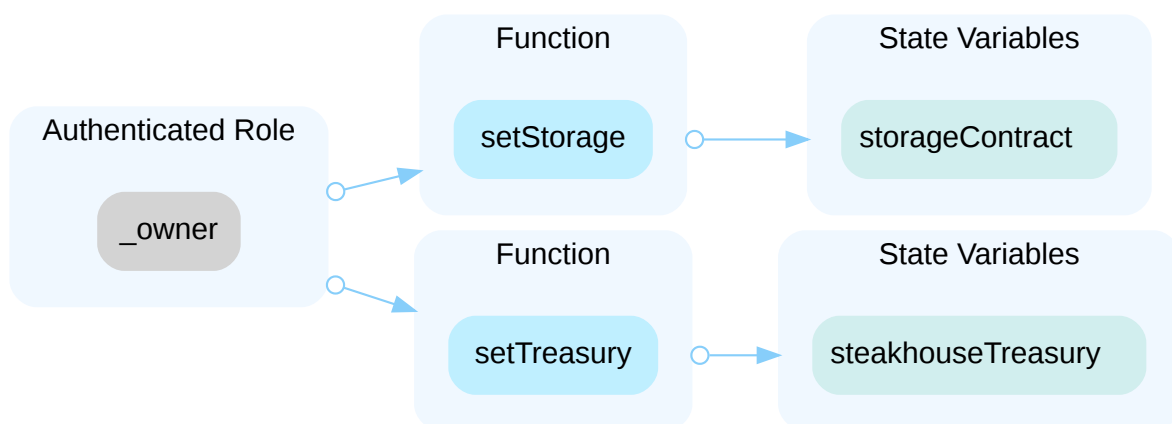
Category	Severity	Location	Status
Centralization	Centralization	contracts/Kitchen.sol (base); contracts/KitchenCreatorBasicAdvanced.sol (base); contracts/KitchenCreatorSimple.sol (base); contracts/KitchenStorage.sol (base); contracts/KitchenUtils.sol (base); contracts/SteakLockers.sol (base)	2/3 Multi-Sig

Description

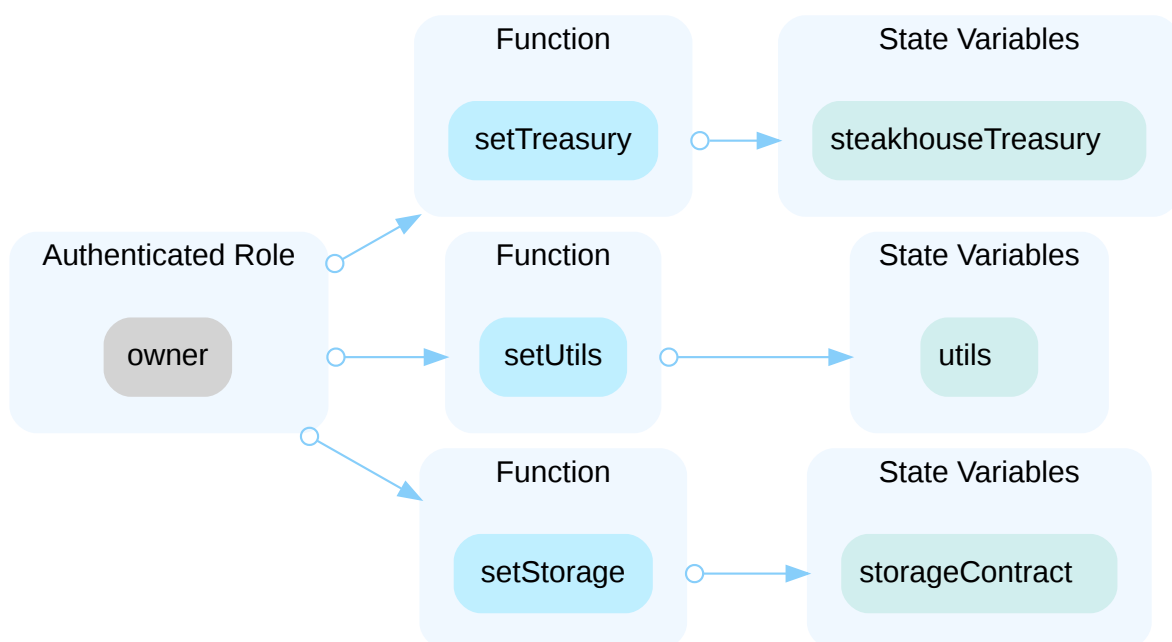
In the contract `Kitchen`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and run a `authorizeAllModules` to whitelist the router, factory, bonding curve, and graduation contracts and reconfigure the factory, bonding curve, graduation module, storage, curve deployer, and direct launcher addresses.



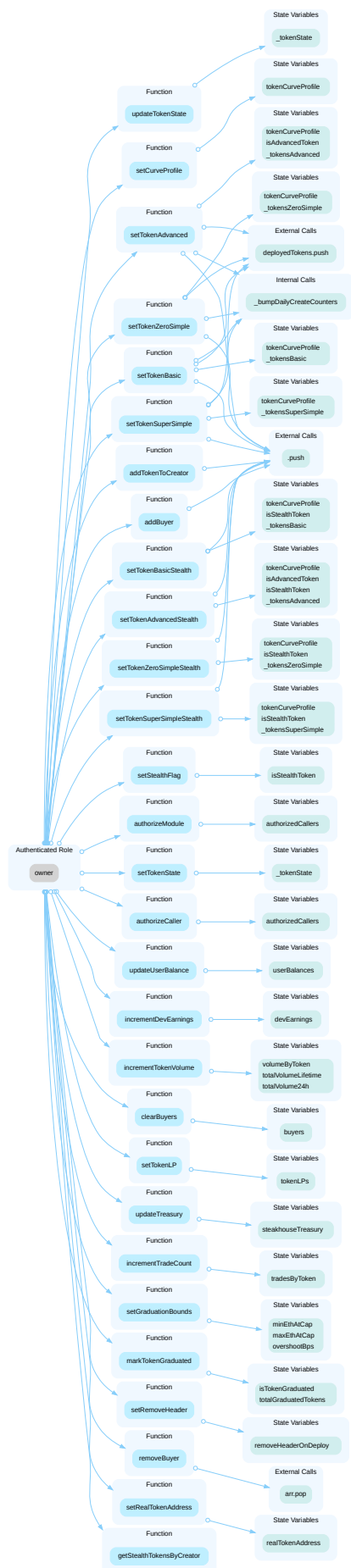
In the contract `KitchenCreatorBasicAdvanced`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and change storage contract and change the treasury recipient.



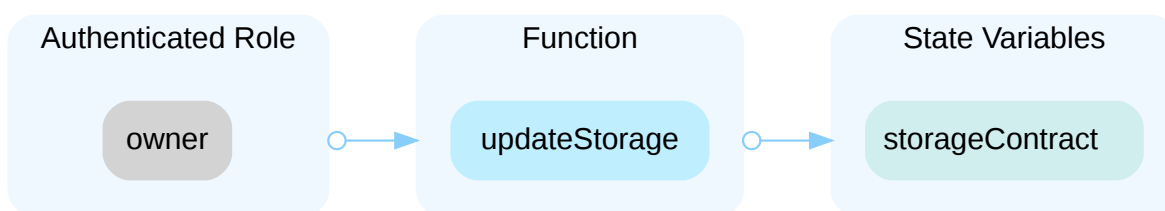
In the contract `KitchenCreatorSimple`, the role `owner` has authority over the functions shown in the diagram below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority and retarget the canonical storage, treasury, and utility helper contracts.



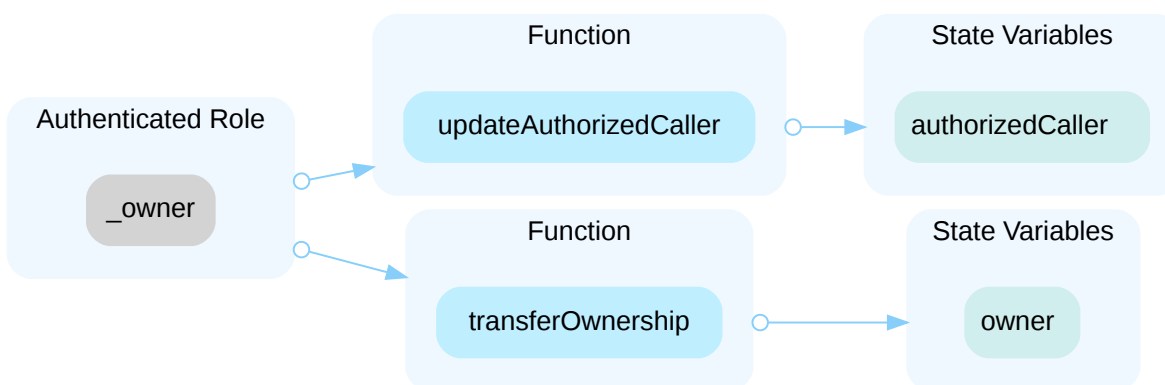
In the contract `KitchenStorage`, the role `owner` has authority over the functions shown in the diagram below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority and can decide who may mutate storage, switch the treasury, and rewrite the system-wide graduation thresholds.



In the contract `KitchenUtils`, the role `owner` has authority over the functions shown in the diagram below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority and repointing all helper reads to a different storage contract through `updateStorage`.



In the contract `SteakLockers`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and changing the minimum lock duration, draining any LP token via `emergencyWithdraw`, rewriting lock expiries, appointing a new privileged caller, or handing ownership to another address.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND

- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

Alleviation

[SteakHouse Finance, 05/08/2026]: The team acknowledged the issue and adopted the multisign solution to ensure the private key management process at the current stage. The below listed contracts have transferred the ownership to a Gnosis Safe contract with 2/3 signers in the sensitive function signing process:

- Storage: [0x5e2D13BaB15C35D5e4FF8f0f3C10351f5Bf10878](#)
- Utils: [0xA79B03120f3f6bd47877afcbe6786336A823Cb83](#)
- Lockers: [0xC22EE6a30dFDcAAE1A47380703544Dd8122fceC5](#)
- CreatorBA: [0xd37667bFF2f58d8276cC270898Ec2AC51Bb05c0a](#)
- CreatorSimple: [0x48CAcE47530357893aA5c2925AeAb24A6a181023](#)
- Kitchen: [0x90f6163E8f0968D949CE11CD031886B65A19b2cc](#)
- Gnosis safe contract address: [0x6c33503Ce90964f22834694742eF9322914D6DF3](#)
- The 3 multisign addresses:
 1. EOA: [0x7B7DE29bc1777E26bb44e8D36a93FDC6b0DaB694](#)
 2. EOA: [0x702d5EebB5ADC9420911ca93d234Cb60a2c313c3](#)

3. EOA:0xE31Fe2f8662aE5c0368903fB20E9efF472d59D2C

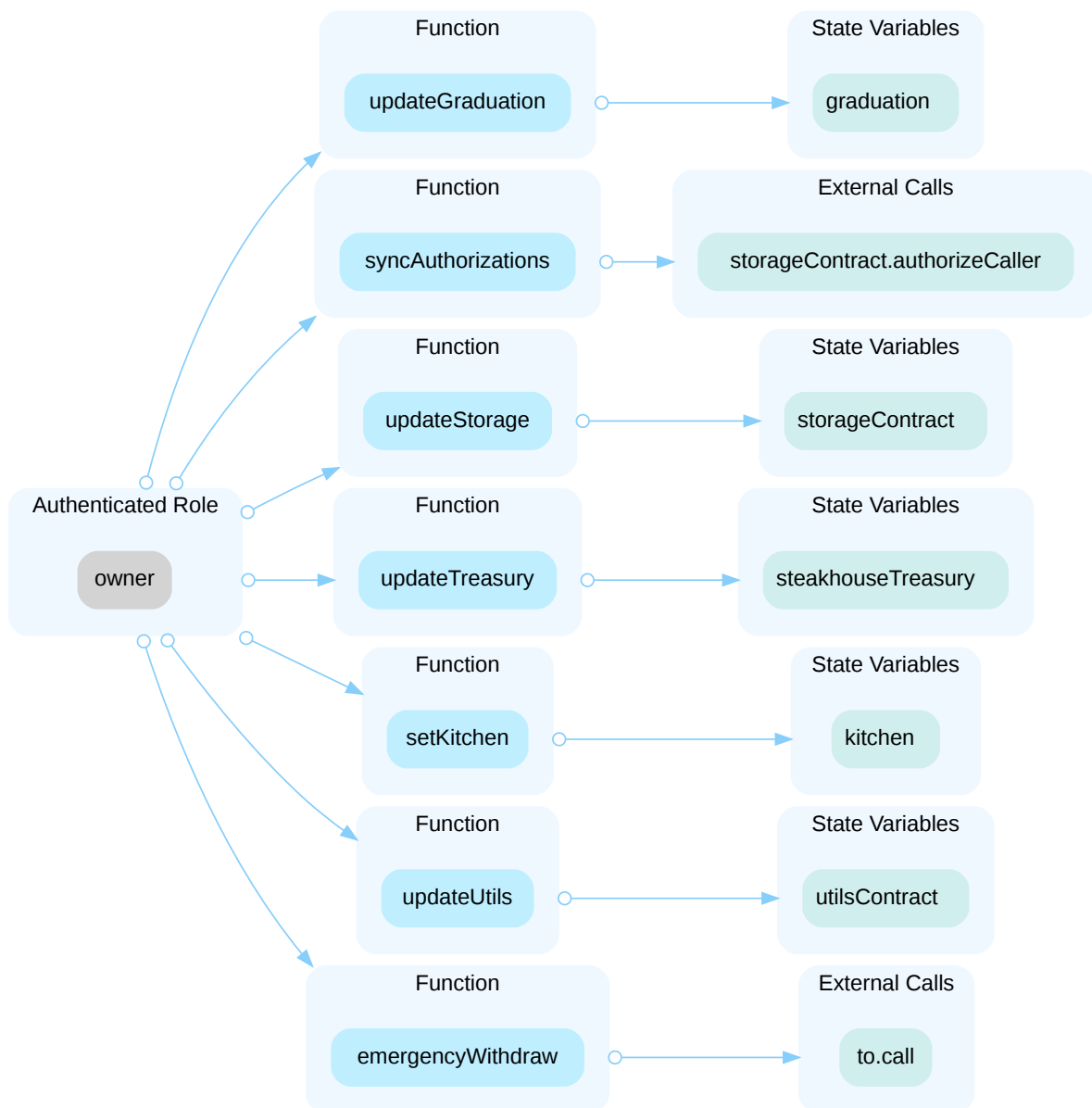
[CertiK, 05/08/2026]: While this strategy has indeed reduced the risk, it's crucial to note that it has not completely eliminated it. CertiK strongly encourages the project team to periodically revisit the private key security management of all above-listed addresses.

SFA-28 | Centralization Risks

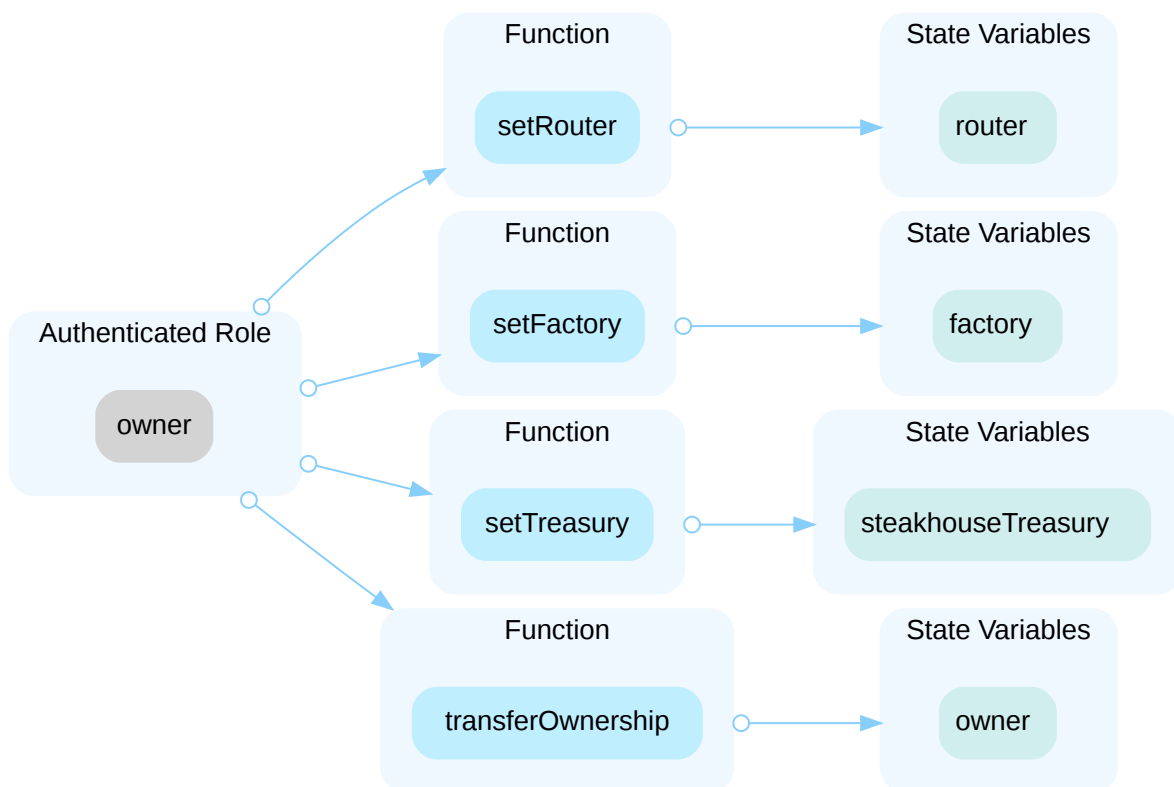
Category	Severity	Location	Status
Centralization	● Centralization	contracts/KitchenBondingCurve.sol (base); contracts/KitchenDeployer.sol (base); contracts/KitchenFactory.sol (base); contracts/KitchenGraduation.sol (base)	● Acknowledged

Description

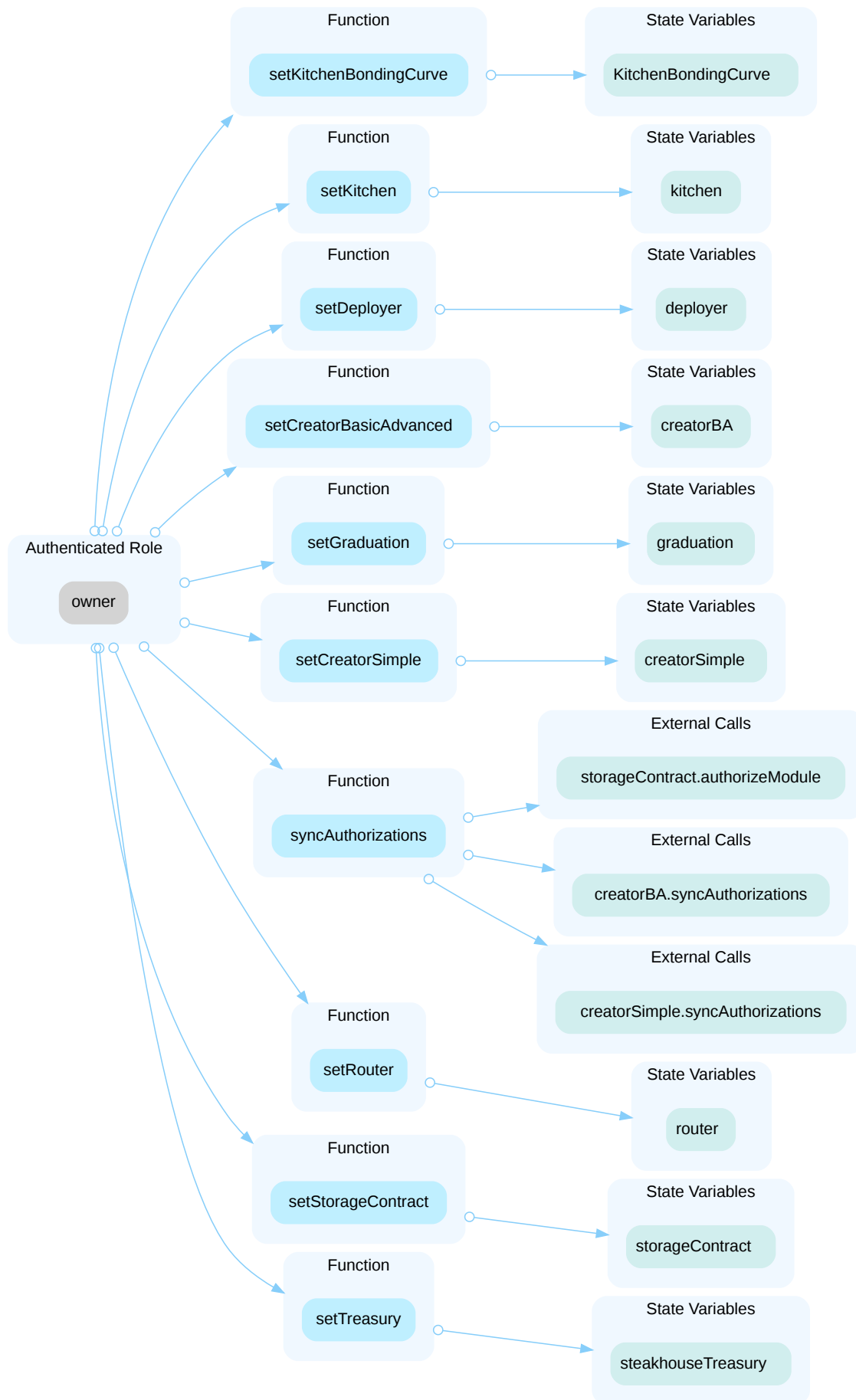
In the contract `KitchenBondingCurve`, the role `owner` has authority over the functions shown in the diagram below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority and resync the storage authorization and retarget the storage, utils, graduation, treasury, and router integrations while also being able to pull ETH from the contract via `emergencyWithdraw`.



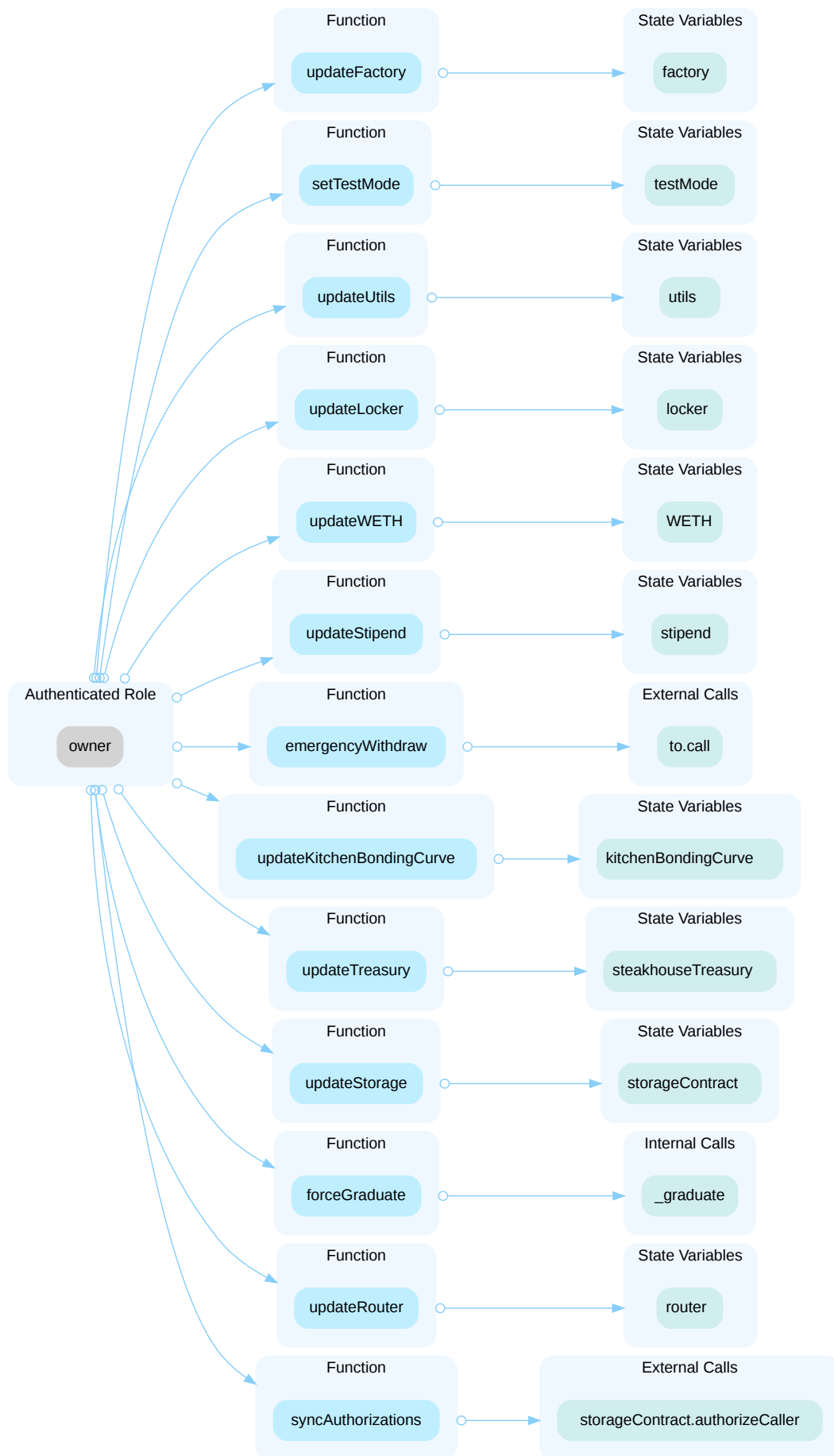
In the contract `KitchenDeployer`, the role `owner` has authority over the functions shown in the diagram below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority and retarget the router, treasury, and factory integrations and transfer ownership.



In the contract `KitchenFactory`, the role `owner` has authority over the functions shown in the diagram below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority and rewire every downstream module via `setCreatorBasicAdvanced`, `setCreatorSimple`, `setDeployer`, `setBondingCurve`, `setRouter`, `setStorageContract`, `setTreasury`, `setKitchen`, and `setGraduation` and can reauthorize all of those components in `KitchenStorage` via `syncAuthorizations`.



In the contract `KitchenGraduation`, the role `owner` has authority over the functions shown in the diagram below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority and toggle `testMode`, force bypassed graduations, adjust the stipend, reauthorize itself with storage, retarget the factory/router/treasury/bonding-curve wiring, and pull ETH out of the contract via `emergencyWithdraw`.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign ($\frac{2}{3}$, $\frac{3}{5}$) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

Alleviation

[SteakHouse Finance, 05/08/2026]: The team acknowledged this issue.

[CertiK, 05/08/2026]: It is suggested to implement the aforementioned methods to avoid centralized failure. Also, CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

SFA-04 | Sell Transaction Amount Incorrectly Capped By Uncirculated Supply

Category	Severity	Location	Status
Logical Issue	Major	contracts/KitchenBondingCurve.sol (base): 260~262; contracts/KitchenCurveMaths.sol (base): 48~69; contracts/KitchenFactory.sol (base): 208~218	Resolved

Description

The `_sell` function calculates the ETH received for selling tokens by calling `KitchenCurveMaths.getEthForTokens`.

Inside `getEthForTokens`, there is an incorrect validation check: `require(tokensSold + tokenIn <= totalSupply, "Over-sell")`. In the context of the `_sell` function, `tokensSold` is the `circulatingSupply` and `tokenIn` is the `amt` of tokens sellback. Therefore, the check becomes `require(circulatingSupply + amt <= totalSupply)`.

First, this design is incorrect. The sellback amt is part of the `circulatingSupply` sold previously.

Second, this logic is flawed as it requires the amount of tokens being sellback (`amt`) to be less than or equal to the amount of tokens not yet in circulation (`totalSupply - circulatingSupply`).

As the `circulatingSupply` increases, the maximum amount of tokens that any user can sellback in a single transaction is progressively reduced.

When the `circulatingSupply` exceeds 50% of the `totalSupply`, users are unable to sellback a great amount of tokens they should be able to sellback. There is no restrictions for `graduationCap` to be greater than 50%.

```
function _enforceCapBounds(
    uint256 totalSupply,
    uint256 graduationCap
) internal view {
    if (graduationCap == 0 || graduationCap >= totalSupply) {
        revert BadCap(graduationCap, totalSupply);
    }
    uint256 ethAtCap = KitchenCurveMaths.ethAtSupplyFromGenesis(
        totalSupply,
        graduationCap
    );
    uint256 minEth = storageContract.minEthAtCap();
    uint256 maxEth = storageContract.maxEthAtCap();
    if (ethAtCap < minEth || ethAtCap > maxEth) {
        revert CapOutOfBounds(ethAtCap, minEth, maxEth);
    }
}
```

This effectively prevents users from selling back large portions of their holdings as the token becomes more popular, trapping their liquidity.

Scenario

Assumed State:

- `totalSupply`: 1,000,000 tokens
- `circulatingSupply`: 950,000 tokens
- A user holds 100,000 tokens.

Steps:

1. The user attempts to sell 60,000 tokens by calling the `sellToken` function with `amt = 60000`.
2. The call to `_sell` will proceed to `getEthForTokens`, where the check `require(circulatingSupply + amt <= totalSupply)` is performed.
3. The check evaluates to `require(950,000 + 60,000 <= 1,000,000)`, which simplifies to `require(1,010,000 <= 1,000,000)`.
4. This condition is false, causing the transaction to revert with the "Over-sell" error message, thus preventing the sale.

Recommendation

The validation check should be corrected to reflect actual circulating dynamics.

Alleviation

[SteakHouse Finance, 10/23/2025]: The team heeded the advice and resolved the issue by removing the check in commit [25bb52b722d1b6c3e3b59021ce4a835d431dd6b6](#)

SFA-06 | HEADERLESS_FEE Payment At Creation Time Can Be Bypassed

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/Kitchen.sol (base): 221~235; contracts/KitchenFactory.sol (base): 224~237; contracts/KitchenGraduation.sol (base): 255, 447~459	● Resolved

Description

The creation fee contains a `HEADERLESS_FEE` which is computed from the user-supplied boolean `removeHeader`, yet that choice can be changed by the creator before deployment.

In the Kitchen contract, the creator can call `setHeaderlessPreference` to flip the choice. Before the deployment, `removeHeader` is just a setup in the storage which does not impact anything.

In the KitchenGraduation contract, the `_resolveRemoveHeader` function reads the `removeHeaderOnDeploy` first and will return true directly if the `removeHeaderOnDeploy` is true. Only if it is false, the function reads the `removeHeader` which is set true via payment.

```
function _resolveRemoveHeader(
    address token,
    KitchenStorage.CurveProfile p
) internal view returns (bool) {
    if (storageContract.removeHeaderOnDeploy(token)) return true;

    if (p == KitchenStorage.CurveProfile.ADVANCED) {
        return storageContract.getTokenAdvanced(token).removeHeader;
    } else if (p == KitchenStorage.CurveProfile.BASIC) {
        return storageContract.getTokenBasic(token).removeHeader;
    } else if (p == KitchenStorage.CurveProfile.SUPER_SIMPLE) {
        return storageContract.getTokenSuperSimple(token).removeHeader;
    } else {
        return storageContract.getTokenZeroSimple(token).removeHeader;
    }
}
```

However, no matter paid in the creation time or not, the creator needs to pay the `HEADERLESS_FEE` again computed by the `_computeGradParams` function to deploy a `realToken` without header.

```

function deployToken(
    string memory name,
    string memory symbol,
    address creator,
    bool isTax,
    bool removeHeader,
    uint256 finalTaxRate,
    uint256 maxSupply
) external payable onlyFactory returns (address tokenAddress) {
    if (isTax && finalTaxRate > 5) revert FinalTaxTooHigh();

    if (removeHeader) { //without header
        if (msg.value < FEE_HEADERLESS) revert HeaderlessFeeTooLow();
        (bool ok, ) = payable(steakhouseTreasury).call{value: msg.value}
(" ");

        require(ok, "headerless fee xfer failed");

        tokenAddress = isTax
            ? address(new TaxToken(name, symbol, maxSupply, finalTaxRate,
creator, creator, router))
            : address(new NoTaxToken(name, symbol, maxSupply));
    } else { //with header SteakHouseImpl
        tokenAddress = isTax
            ? address(new TaxTokenSteakHouseImpl(name, symbol, maxSupply,
finalTaxRate, creator, creator, router))
            : address(new NoTaxTokenSteakHouseImpl(name, symbol,
maxSupply));
    }
    return tokenAddress;
}

```

This design makes the HEADERLESS_FEE paid in the creation time meaningless. Creators without HEADERLESS_FEE payment in the creation time can flip their choices via `setHeaderlessPreference`.

Recommendation

Unify the fee logic:

- Remove the `setHeaderlessPreference` function so that the `headerless` configuration cannot be changed.
- Or remove the creation-time HEADERLESS_FEE entirely, enforcing the fee only at graduation (`deployToken`).

Alleviation

[SteakHouse Finance, 10/24/2025]: The team heeded the advice and resolved the issue in commit

[8a8c3b1019261255573d8a70eb3387264740477e](#)

SFA-07 | Improper Conditioning Of `finalTaxRate` Validation Allows Tax Limit Bypass

Category	Severity	Location	Status
Inconsistency	● Medium	contracts/KitchenCreatorBasicAdvanced.sol (base); contracts/KitchenDeployer.sol (base)	● Resolved

Description

According to the logic of function `deployToken`, the `tokenType` value `isTax` determines which contract would be deployed: `TaxToken` or `NoTaxToken`.

```
function deployToken(
    string memory name,
    string memory symbol,
    address creator,
    bool isTax,
    bool removeHeader,
    uint256 finalTaxRate,
    uint256 maxSupply
) external payable onlyFactory returns (address tokenAddress) {
    if (isTax && finalTaxRate > 5) revert FinalTaxTooHigh();

    if (removeHeader) {
        if (msg.value < FEE_HEADERLESS) revert HeaderlessFeeTooLow();
        (bool ok, ) = payable(steakhouseTreasury).call{value: msg.value}("");
        require(ok, "headerless fee xfer failed");

        tokenAddress = isTax
            ? address(new TaxToken(name, symbol, maxSupply, finalTaxRate,
creator, creator, router))
            : address(new NoTaxToken(name, symbol, maxSupply));
    } else {
        tokenAddress = isTax
            ? address(new TaxTokenSteakHouseImpl(name, symbol, maxSupply,
finalTaxRate, creator, creator, router))
            : address(new NoTaxTokenSteakHouseImpl(name, symbol, maxSupply));
    }
    return tokenAddress;
}
```

However, the validation for `finalTaxRate` is improperly conditioned on the `tokenType`. The check `if (finalTaxRate > 5)` is only performed for tokens explicitly marked `isTax` as true. The same validation logic is applied when creating tokens for basic and advanced tokens:

```
if (b.tokenType == KitchenStorage.TokenType.TAX && b.finalTaxRate > 5)
    revert FinalTaxTooHigh();
```

The `finalTaxRate` is a factor to determine the result of `getCurrentTax`. No matter `isTax` true or false, the `devPerc` is charged in the buy/sell for basic and advanced tokens.

```

function _buy(address token, address buyer, uint256 ethIn) internal {
    ...
    uint256 devPerc = _getCurrentDevTaxPercent(token);
    ...
}
function getCurrentTax(address token) public view returns (uint256) {
    KitchenStorage.CurveProfile p = storageContract.tokenCurveProfile(
        token
    );

    if (p == KitchenStorage.CurveProfile.ADVANCED) {
        KitchenStorage.TokenAdvanced memory a = storageContract
            .getTokenAdvanced(token);
        KitchenStorage.TokenState memory s = storageContract.getTokenState(
            token
        );

        if (a.taxDropInterval == 0) return a.curveStartingTax;

        uint256 steps = (block.timestamp - s.createdAtTimestamp) /
            a.taxDropInterval;
        uint256 decay = steps * a.taxDropStep;

        if (decay >= a.curveStartingTax) return a.finalTaxRate;
        uint256 current = a.curveStartingTax - decay;
        return current < a.finalTaxRate ? a.finalTaxRate : current;
    }

    // BASIC profile: static startingTax that flips to finalTax after
    // `curveTaxDuration` seconds
    if (p == KitchenStorage.CurveProfile.BASIC) {
        KitchenStorage.TokenBasic memory b = storageContract.getTokenBasic(
            token
        );
        KitchenStorage.TokenState memory s = storageContract.getTokenState(
            token
        );
        if (b.curveTaxDuration == 0) return b.curveStartingTax;
        bool finished = block.timestamp >=
            s.createdAtTimestamp + b.curveTaxDuration;
        return finished ? b.finalTaxRate : b.curveStartingTax;
    }

    return 0; // SUPER_SIMPLE or ZERO_SIMPLE
}

```

This allows a creator to bypass the tax rate limit by setting the `tokenType` to false while simultaneously setting a `finalTaxRate` far exceeding the intended maximum of 5%. Related functions with this issue:

- `createBasicToken()`
- `createBasicTokenStealth()`
- `createAdvancedToken()`
- `createAdvancedTokenStealth()`

More significantly, in the `graduate` it checks if the final tax rate is smaller than 5, without the validation, it can cause the token never be able to graduate.

Recommendation

- Either not depend on `tokenType` for the validation of `finalTaxRate`.
- Or add the validation of `tokenType` in the `utilsContract.getCurrentTax`.

Alleviation

[SteakHouse Finance, 10/24/2025]: The team heeded the advice and resolved the issue in commit [2aecf49ed23e645b639e7cfd96a2f93b6a2917e6](#)

SFA-08 | Unbounded Loop In `_airdropBuyers` And Missing Stipend Funding

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/KitchenGraduation.sol (base): 376~386	● Resolved

Description

The `_airdropBuyers` function iterates over the entire `buyers` array and mints tokens to each address in a single transaction:

```
function _airdropBuyers(address virtualToken, address realToken) private {
    address[] memory arr = storageContract.getBuyers(virtualToken);
    uint256 len = arr.length;
    for (uint256 i = 0; i < len; i++) {
        address u = arr[i];
        uint256 amt = storageContract.getUserBalance(u, virtualToken);
        if (amt > 0) {
            IKitchenFactory(factory).mintRealToken(realToken, u, amt);
        }
    }
}
```

This design introduces two problems:

1. Gas Limit and DoS Risk

- If `buyers.length` is large (e.g., thousands of addresses), the loop may exceed the block gas limit or become prohibitively expensive.
- This can prevent graduation from completing, effectively locking the system in a non-graduated state.

2. Uncompensated Caller Cost

- `_airdropBuyers` is invoked synchronously within `_graduate`, meaning the final caller must pay gas to mint for *all* buyers.
- The contract attempts to refund a stipend:

```
// This is paid from contract balance after _pullFromCurve.
if (stipend > 0 && address(this).balance >= stipend && buyer != address(0)) {
    (bool ok, ) = payable(buyer).call{value: stipend}("");
    require(ok, "Gas refund xfer failed");
}
```

- However, `_pullFromCurve` only retrieves `g.totalFee + g.ethForLP` and does not include the stipend amount.
- Although the code comments in the curve says allow a stipend refund, it is not implemented. Then the stipend may be unfunded, leaving the caller uncompensated despite covering a potentially enormous gas cost.

```
// stipend margin reserved for graduation gas refund
// Stipend reserved to refund the buyer who triggers graduation (paid by
// Graduation after pulling ETH from the curve).
uint256 public constant STIPEND = 0.003 ether;

// Auto-graduation: if cap reached (allowing configured overshoot), call
// into Graduation. The last buyer will be passed as `buyer` to allow a
// stipend refund.

uint256 overshootCap = st.cap +
    (st.cap * storageContract.overshootBps()) /
    10_000;
if ((st.circ + L.tokensOut) >= st.cap) {
    require(
        (st.circ + L.tokensOut) <= overshootCap,
        "Exceeds overshoot limit"
    );
    // auto-graduate
    IKitchenGraduation(graduation).graduateToken(token, buyer);
}
```

As a result, the graduation can become non-executable (DoS) for tokens with large buyer sets. Callers have no incentive to trigger graduation since they bear all gas costs without guaranteed reimbursement.

Recommendation

We recommend:

- Replace bulk airdrop with claim-based distribution.
- Fix stipend funding.

Alleviation

[SteakHouse Finance, 11/07/2025]: The team heeded the advice and resolved the issue by funding the stipend in commit [c38f4e0fa27db00b46f645c2972b6255a699460e](https://github.com/steakhouse-finance/steakhouse-finance/commit/c38f4e0fa27db00b46f645c2972b6255a699460e).

SFA-19 | Exceeding `maxEthAtCap` May Revert

Category	Severity	Location	Status
Design Issue	● Medium	contracts/KitchenGraduation.sol (base): 251	● Resolved

Description

The graduation mechanism enforces a hard upper bound `maxEthAtCap` = 6.6 ether. Under the current design of the bonding curve with overshoot allowance (`overshootBps` up to 5000), it is mathematically inevitable that in some parameter ranges a final buy can push cumulative ETH in the pool above this limit.

Specifically:

The function `_enforceCapBounds` requires `ethAtCap` between 1.6 and 6.6 ether.

However, when overshoot is large, any valid choice of cap and totalSupply will still allow a “legal overshoot” where `circulatingSupply + tokensOut ≤ cap + cap*overshootBps/10000`, yet the cumulative ETH `ethRaised + ethIn` is larger than 6.6.

In such a case, the system has no configurations to satisfy both (a) the overshoot allowance and (b) the ETH graduation limit. When triggered on-chain, the transaction will revert.

Users attempting to buy near graduation will face guaranteed reverts, resulting in poor UX and potentially bricked pools.

Protocol operators cannot configure parameters to avoid the condition if high overshoot values are permitted.

This creates a mismatch between design intent (“allow overshoot up to 5000 bps”) and system invariants (“ETH ≤ 6.6”).

Scenario

1. We assume that some parameters of token A are as below:

`totalSupply` = 10,000

`cap` = 8,000

`overshootBps` = 2000 (20%)

- 2.State before last buy:

`circulatingSupply` = 7,600

`ethRaised` ≈ `ethAtSupplyFromGenesis(10000, 7600)` = $7,600 / (10,000 - 7,600) \approx 3.1667$ ETH

- 3.Final buy:

`ethIn` = 5.0 ETH

$\text{tokensOut} = \text{getTokensForEth}(\dots) \approx 1,309.09$

New circulating supply = $7,600 + 1,309.09 \approx 8,909.09$

4. Checks:

Overshoot still valid:

$\text{Limit} = \text{cap} + \text{cap} * \text{overshoot} / 10000 = 8,000 * 1.2 = 9,600$

$8,909.09 \leq 9,600$

However, the accrued ETH after buy:

$3.1667 + 5.0 = 8.1667 \text{ ETH}$

$8.1667 > 6.6$

Thus the transaction satisfies the overshoot allowance but violates the ETH graduation bound, causing the buy transaction to inevitably revert.

■ Recommendation

We recommend the team to confirm if it the intended design and perform corresponding modifications to ensure project correct operations.

■ Alleviation

[SteakHouse Finance, 10/27/2025]: The team heeded the advice and resolved the issue in commit [a22600b62cff37490721993929f91de11c031bf8](#).

SFA-22 | Lack Of tokenId Uniqueness Checks In KitchenCreatorBasicAdvanced

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/KitchenCreatorBasicAdvanced.sol (base): 91~96; contracts/KitchenCreatorSimple.sol (base): 54~59, 73~79; contracts/KitchenStorage.sol (base): 304	● Resolved

Description

The `KitchenCreatorSimple` contract includes strict uniqueness checks in `_checksUnique(address token)`, ensuring that a generated token address has not already been registered in any of the token registries (Basic, Advanced, SuperSimple, ZeroSimple). If a token already exists, the function reverts. However, the `KitchenCreatorBasicAdvanced` contract does not implement such uniqueness checks. Both `KitchenCreatorSimple` and `KitchenCreatorBasicAdvanced` generate token IDs using the same deterministic scheme, with a contract-local `_nonce`:

```
function _newVirtualTokenId(address creator) internal returns (address token) {
    _nonce++;
    token = address(uint160(uint256(keccak256(
        abi.encodePacked(creator, _nonce)
    ))));
}
```

Token states from both creators are ultimately stored in a shared contract `KitchenStorage`:

```
function setTokenState(address token, TokenState memory data) external
onlyAuthorized {
    _tokenState[token] = data;
}
```

Since `KitchenCreatorBasicAdvanced` does not check for existing tokens, an attacker can:

1. Create a new token in `KitchenCreatorSimple`, storing its state in `KitchenStorage`.
2. Wait until the same creator and matching `_nonce` value in `KitchenCreatorBasicAdvanced` produce an identical token address.
3. During the token creation, the function `setTokenState` overwrites the previous token state in `KitchenStorage`.

Recommendation

We recommend the team to implement similar uniqueness check mechanism in `KitchenCreatorBasicAdvanced` contract.

I Alleviation

[SteakHouse Finance, 10/22/2025]: The team heeded the advice and resolved the issue in commit [3884f77b79a2a514bc09d2ada8b748b22b602770](#).

SFA-23 | StealthFee Is Not Paid In `createSuperSimpleToken` And `createZeroSimpleToken`

Category	Severity	Location	Status
Logical Issue	● Medium		● Resolved

Description

The `createSuperSimpleToken` and `createZeroSimpleToken` functions fail to charge the `STEALTH_FEE` when a user requests the creation of a stealth token. Both functions accept a boolean parameter `isStealth` which is passed to the underlying token creation module. However, when invoking the internal `_collectFees` function to calculate and transfer the required fees, they hardcode the `isStealth` argument to `false`. This logical disconnect allows a user to create a stealth token while only paying the base fee, causing a loss of protocol revenue for the treasury.

Scenario

Exploit Demonstration:

1. A user wants to create a 'Super Simple' stealth token. The correct total fee for this action should be `BASE_FEE_SUPERSIMPLE` (0.001 ether) + `STEALTH_FEE` (0.003 ether), totaling 0.004 ether.
2. The user calls the `createSuperSimpleToken` function, providing the necessary token metadata and setting the `isStealth` parameter to `true`.
3. Instead of sending the correct 0.004 ether, the user sends only 0.001 ether with the transaction (`msg.value = 0.001 ether`).
4. The `createSuperSimpleToken` function internally calls `_collectFees`, but with its `isStealth` argument hardcoded to `false`.
5. The `_collectFees` function calculates the required fee as 0.001 ether, which matches the `msg.value` sent by the user. The fee check passes, and 0.001 ether is transferred to the treasury.
6. The execution proceeds, and the `creatorSimple.createSuperSimpleToken` function is called with the user's original `isStealth = true` parameter, resulting in the successful creation of a stealth token.
7. The user has successfully created a stealth token while evading the 0.003 ether `STEALTH_FEE`.

Recommendation

Instead of hardcode it to false, use the `isStealth` argument.

Alleviation

[SteakHouse Finance, 10/22/2025]: The team heeded the advice and resolved the issue in commit [94f06c0817d23357ed7a6d8f7fca27890d14e2d1](#).

SFA-26 Anti-Whale Limits Can Be Bypassed Due To Time Base Mismatch

Category	Severity	Location	Status
Logical Issue	Medium	contracts/KitchenBondingCurve.sol (base); contracts/KitchenUtils.sol (base)	Resolved

Description

Trading is gated by `st.startTime` in `_buy` (`require(block.timestamp >= st.startTime)`), but the max wallet/tx limit schedules (`getCurrentMaxWallet/getCurrentMaxTx` and `isLimitsLifted`) advance using `storageContract.getTokenState(token).createdAtTimestamp`, not `startTime`.

If the input parameter `startTime` is set significantly later than `createdAtTimestamp(current time)`, the limit durations elapse before trading is allowed.

```
function createBasicToken(
    BasicParamsBasic calldata b,
    StaticCurveParams calldata s,
    address creator
) external payable {
    ...
    _pushInitialState(token, b.startTime);
    ...
}

function _pushInitialState(address token, uint256 startTime) internal {
    KitchenStorage.TokenState memory st = KitchenStorage.TokenState({
        ethPool: 0,
        circulatingSupply: 0,
        graduated: false,
        createdAtBlock: block.number,
        createdAtTimestamp: block.timestamp,
        startTime: startTime
    });
    storageContract.setTokenState(token, st);
}
```

At the moment trading opens, limits may already have lifted (become extremely large), allowing an immediate large buy that defeats intended early-phase protections.

```
function isLimitsLifted(address token) public view returns (bool) {
    KitchenStorage.CurveProfile p = storageContract.tokenCurveProfile(
        token
    );
    //ADVANCED:
    if (p == KitchenStorage.CurveProfile.ADVANCED) {
        KitchenStorage.TokenAdvanced memory a = storageContract
            .getTokenAdvanced(token);
        KitchenStorage.TokenState memory s = storageContract.getTokenState(
            token
        );
        return block.timestamp >= s.createdAtTimestamp + a.limitRemovalTime;
    }
    ...
}
```

The function enforces liveness using `startTime` but enforces limits using `createdAtTimestamp`, creating inconsistent time bases. Limits intended to protect the initial trading window can be silently neutralized before trading actually starts.

Scenario

- Identify a token where `createdAtTimestamp` was set long before `startTime`, and the configured limit durations (`limitRemovalTime`, `curveMaxWalletDuration`, `curveMaxTxDuration`, etc.) are shorter than the gap.
- Wait until `block.timestamp >= startTime`.
- Call the buy entrypoint once trading opens. Because `getCurrentMaxWallet/getCurrentMaxTx` base their schedules on `createdAtTimestamp`, limits will already be effectively unbounded or very high.
- Buy a very large amount in a single transaction (potentially the entire remaining cap within overshoot rules), bypassing the an

Recommendation

Use a dedicated `limitsStart` as the time base for limit scheduling and expiry.

Alleviation

[SteakHouse Finance, 10/27/2025]: The team heeded the advice and resolved the issue in commit

[81768f0b5338cd70e9a343694bc2e4214d6cc7d0](#)

SFA-01 | Liquidity Pool Price Manipulation During Token Graduation

Category	Severity	Location	Status
Design Issue	● Minor	contracts/KitchenGraduation.sol (base): 344~346	● Resolved

Description

The `_computeGradParams` function calculates the amount of ETH (`g.ethForLP`) and tokens (`g.tokensToLP`) to be added to the liquidity pool upon a token's graduation. The calculation for `g.tokensToLP` is based on the `finalPrice` of the token. The function includes a check to ensure the number of tokens to be minted does not exceed the token's `totalSupply`. If the calculated `g.tokensToLP` exceeds the remaining available supply, it is capped (reduced).

However, as a result, the added pool price will be raised due to the capped tokens, which would provide an arbitrage opportunity.

Recommendation

Review the logic and fix the issue.

Alleviation

[SteakHouse Finance, 10/23/2025]: Instead of seeding the lp with all eth and inflating token price in the pool and creating a arbitrage opportunity to sell into, therefore creating a negative starting price on the chart I have instead created a split for the eth with the exact being seeded into lp to create the exact price the curve ends on and have created a eth injection mechanism for the left over eth to buyback and burn the live token with the remaining eth creating the 1st organic candle and price action on launch within the same graduation flow following adding liquidity.

Commit [4b311f1d6b827f58403f1d4bade4a959c9e991e7](#)

SFA-09 | Unchecked ERC-20 `transfer()` / `transferFrom()` Call

Category	Severity	Location	Status
Volatile Code	Minor	contracts/KitchenBondingCurve.sol (base): 393~423; contracts/SteakLoc kers.sol (base): 144~148	Resolved

Description

The return values of the `transfer()` and `transferFrom()` calls in the smart contract are not checked. Some ERC-20 tokens' transfer functions return no values, while others return a bool value, they should be handled with care. If a function returns `false` instead of reverting upon failure, an unchecked failed transfer could be mistakenly considered successful in the contract.

```
416 IERC20(lpToken).transfer(DEAD, liquidity);
```

```
147 IERC20(lpToken).transfer(to, bal);
```

Recommendation

It is advised to use the OpenZeppelin's `SafeERC20.sol` implementation to interact with the `transfer()` and `transferFrom()` functions of external ERC-20 tokens. The OpenZeppelin implementation checks for the existence of a return value and reverts if false is returned, making it compatible with all ERC-20 token implementations.

Alleviation

[SteakHouse Finance, 10/27/2025]: The team heeded the advice and resolved the issue by in commit [1063474d0dcc1240706adc47d6e70b42dabd02fd](#)

SFA-10 | Unused Return Value

Category	Severity	Location	Status
Volatile Code	Minor	KitchenGraduation.sol: 393~423	Resolved

Description

The smart contract does not check or store the return value of an external call in a local or state variable, which may introduce vulnerabilities due to the unhandled outcome.

```
404      (, , liquidity) = IUniswapV2Router02(router).addLiquidityETH{value:
ethForLP}(
405          realToken,
406          tokensForLP,
407          0,
408          0,
409          address(this),
410          block.timestamp
411      );
```

```
402      IERC20(realToken).approve(router, tokensForLP);
```

```
419      IERC20(lpToken).approve(locker, liquidity);
```

Recommendation

It is suggested to ensure proper error handling by checking or using the return values of all external function calls, and storing them in appropriate local or state variables if necessary.

Alleviation

[SteakHouse Finance, 10/27/2025]: The team heeded the advice and resolved the issue by in commit [f6009b9cab8c494eb656cc408705d8ca488752dd](#)

SFA-11 | Unchecked Unlock Time

Category	Severity	Location	Status
Logical Issue	● Minor	contracts/SteakLockers.sol (base): 161~162	● Resolved

Description

In the function `forceUpdateUnlock`, the input parameter of `newUnlockTime` is not checked to be greater than the current `block.timestamp`. In this way, the owner could use this function to unlock the lpToken rather than update the unlock time.

Recommendation

We recommend ensuring that the lock is created with a reasonable unlocking delay.

Alleviation

[SteakHouse Finance, 10/27/2025]: The team heeded the advice and resolved the issue by removing the function `forceUpdateUnlock()` in commit [4a2e0ec130bef104e16eb53f1c647f5e1db6607f](#).

SFA-12 Payable Creation Functions Can Unintentionally Lock Native Tokens

Category	Severity	Location	Status
Logical Issue	Minor	contracts/KitchenFactory.sol (base)	Resolved

Description

The KitchenFactory contract calls several payable creation functions and forward the `msg.value - required` amount of ETH, but creation functions never use `msg.value` and provides no withdrawal methods. Any ETH sent to these functions will be accepted by the contract and become permanently stuck.

```
creatorBA.createBasicToken{value: forward}(b, s, creator);
```

Similar payable functions:

- `createBasicToken()`
- `createBasicTokenStealth()`
- `createAdvancedToken()`
- `createAdvancedTokenStealth()`
- `createSuperSimpleToken()`
- `createZeroSimpleToken()`

Recommendation

Consider adding a withdrawal path and/or reverting on nonzero `msg.value - required`.

Alleviation

[SteakHouse Finance, 10/27/2025]: The team heeded the advice and resolved the issue in commit [951f220143a1dbc7bd6cc0c68796981035668ba8](#).

SFA-13 | Gas-DoS On Sells Due To $O(N)$ Linear Scan In `removeBuyer`

Category	Severity	Location	Status
Logical Issue	Minor	contracts/KitchenGraduation.sol (base)	Resolved

Description

When a sell reduces the seller's balance to zero, `_sell` calls `storageContract.removeBuyer(token, seller)`. That function linearly scans `buyers[token]` to find and remove the address.

```
function removeBuyer(address token, address user) external onlyAuthorized {
    if (userBalances[user][token] == 0) {
        address[] storage arr = buyers[token];
        uint256 len = arr.length;
        for (uint256 i = 0; i < len; ++i) {
            if (arr[i] == user) {
                arr[i] = arr[len - 1];
                arr.pop();
                break;
            }
        }
    }
}
```

This $O(n)$ loop scales with the number of unique buyers for the token. For sufficiently large buyer sets, calling `_sell` with full-balance can exceed the block gas limit and revert, effectively DoSing full-balance sells for addresses—particularly those located late in the buyers array.

Scenario

1. Ensure `buyers[token]` is large (this is common for popular tokens; the attacker can also bloat it by many small unique buys prior to the attack).
2. From a buyer address near the end of the buyers array, call `_sell(token, seller, amt)` with `amt == userBal` so that `newBal == 0`.
3. The call to `removeBuyer(token, seller)` iterates over the entire buyers array until it finds the seller. With a large enough array, the transaction runs out of gas and reverts.
4. Result: full-balance sells become infeasible (DoS) for affected users until the buyers array shrinks or gas costs change, purely due to the linear removal logic triggered by this function.

Recommendation

Use mappings or lazy removal.

I Alleviation

[SteakHouse Finance, 10/27/2025]: The team heeded the advice and resolved the issue in commit 30ff032b80c857bf5cd6637493d331f53d1130fb.

SFA-14 | Unnecessary Access In `deployToken`

Category	Severity	Location	Status
Access Control	Minor	contracts/KitchenFactory.sol (base): 316	Resolved

Description

The `deployToken` function enforces that only the `graduation` or `bondingCurve` contracts may call it:

```
function deployToken(
    string memory name,
    string memory symbol,
    address creator,
    bool isTax,
    bool removeHeader,
    uint256 finalTaxRate,
    uint256 maxSupply
) external payable returns (address tokenAddress) {
    if (msg.sender != graduation && msg.sender != bondingCurve)
        revert OnlyCurve();
    return
        deployer.deployToken{value: msg.value}(
            name,
            symbol,
            creator,
            isTax,
            removeHeader,
            finalTaxRate,
            maxSupply
        );
}
```

However, the codebase shows that only the `graduation` contract actually invokes this function. For example, `KitchenGraduation._deployRealToken` always calls `IKitchenFactory(factory).deployToken(...)`, and the `bondingCurve` contract never does.

This makes the `bondingCurve` branch in the access restriction unnecessary.

Recommendation

Restrict `deployToken` exclusively to the `graduation` contract, since `bondingCurve` does not call it.

Alleviation

[SteakHouse Finance, 10/28/2025]: The team heeded the advice and resolved the issue in commit
[fa7d10c6a2be9fd195fc33278af110668299224f](#)

SFA-18 | Potential Sandwich Attacks In Bonding Curve

Category	Severity	Location	Status
Logical Issue	● Minor	contracts/KitchenBondingCurve.sol (base): 166	● Resolved

Description

A sandwich attack can occur when an attacker notices a transaction involving token swaps or liquidity additions that lacks slippage restrictions or minimum output amount settings. The attacker can exploit this by frontrunning the transaction to purchase one of the assets, thereby manipulating the exchange rate, and then backrunning the transaction to sell the asset for a profit. The referenced function is invoked without setting slippage restrictions or minimum output amounts, making transactions that trigger these functions particularly vulnerable to sandwich attacks, especially when large input amounts are involved.

Recommendation

We recommend setting reasonable minimum output amounts, instead of 0, based on token prices when calling the aforementioned functions. Since the price cannot be determined during the swap (e.g., not available from an external price oracle such as Chainlink), it's recommended to limit the buy amount in one transaction.

Alleviation

[SteakHouse Finance, 10/22/2025]: The team heeded the advice and resolved the issue in commit [683c192608d43a1d457ddb504a06a7bdd4f7c290](#).

SFA-24 | Possible Address Collision In `KitchenCreatorBasicAdvanced` And `KitchenCreatorSimple`

Category	Severity	Location	Status
Denial of Service	● Minor	contracts/KitchenCreatorBasicAdvanced.sol (base): 92~95	● Resolved

Description

Since the nonce in `KitchenCreatorSimple` and `KitchenCreatorBasicAdvanced` are maintained separately, when the nonce in those two contracts are the same, the user may not be able to create a token if he creates one in one of the contract. He has to wait another user creates a token before he is able to doing so.

Recommendation

Use the same nonce variable in both contracts, possibly maintains the variable in the storage contract.

Alleviation

[SteakHouse Finance, 10/22/2025]: The team heeded the advice and resolved the issue in commit [359f1b41d2a6ed6c7b89f81d257737a98619b7dc](#).

SFA-25 | Lack Of Input Validation

Category	Severity	Location	Status
Volatile Code	● Minor		● Resolved

Description

Across the project there is a general lack of input validation for parameters when creating tokens such as: `totalSupply`, `maxWallet`, `sellAmount`, etc.

In many cases, these values are taken directly from user input or constructor arguments with no checks on minimum or maximum bounds.

For example: If `totalSupply` is set to an unusually small value, then `getTokensForEth` or `getEthForTokens` can return very small token/ETH amounts, where integer division and rounding down will cause loss of precision. Those small-scale operations impose a risk of economic loss for users if inputs are not constrained.

Recommendation

We recommend the team to implement input validations for all critical parameters.

Alleviation

[SteakHouse Finance, 11/03/2025]: The team heeded the advice and resolved the issue in commit [858b9e60cd0fa43653b63bfbfe7d3076ec5087ac](https://github.com/steakhouse-finance/steakhouse-finance/commit/858b9e60cd0fa43653b63bfbfe7d3076ec5087ac).

SFA-27 | Cross-Function Reentrancy Between `buyToken` And `sellToken`

Category	Severity	Location	Status
Inconsistency	● Minor	contracts/KitchenBondingCurve.sol (42797c): 120~122, 126~128, 230~233, 338~339	● Resolved

Description

The contract's `buyToken` and `sellToken` functions both send ETH to external addresses (`buyer` , `seller` , `treasury` and `payee`) using low-level call. These calls can trigger arbitrary fallback functions, enabling **reentrancy**.

While `sellToken` is protected with the `nonReentrant` modifier, `buyToken` is **not**. This asymmetry allows a malicious receiver (or compromised treasury and payee) to reenter from one function into the other:

- **Scenario 1:** During a `buyToken` call, ETH is sent to a `buyer` whose fallback reenters `sellToken`.
- **Scenario 2:** During a `sellToken` call, ETH is sent to a `seller` , whose fallback reenters `buyToken`.

Recommendation

Apply `nonReentrant` to both `buyToken` and `sellToken` .

Alleviation

[SteakHouse Finance, 11/05/2025]: The team heeded the advice and resolved the issue in commit [0832dd752c5032ba508a2ef4b8f926d647ed60f0](#).

SFA-15 | Divide Before Multiply

Category	Severity	Location	Status
Incorrect Calculation	● Informational	KitchenUtils.sol: 44~71, 100~127, 129~155	● Resolved

Description

Solidity's integer division truncates. Thus, performing division before multiplication can lead to precision loss.

```
137         uint256 steps = (block.timestamp - s.createdAtTimestamp) / a.  
maxTxInterval;
```

```
138         return a.maxTxStart + (steps * a.maxTxStep);
```

```
53         uint256 steps = (block.timestamp - s.createdAtTimestamp) / a.  
taxDropInterval;
```

```
54         uint256 decay = steps * a.taxDropStep;
```

```
108        uint256 steps = (block.timestamp - s.createdAtTimestamp) / a.  
maxWalletInterval;
```

```
109        return a.maxWalletStart + (steps * a.maxWalletStep);
```

Recommendation

We recommend applying multiplication before division to avoid loss of precision.

Alleviation

[SteakHouse Finance, 10/28/2025]: The team heeded the advice and resolved the issue in commit [df31ae35aeb3eef41f0bb1f7461812928573199f](https://github.com/steakhouse/steakhouse-finance/commit/df31ae35aeb3eef41f0bb1f7461812928573199f)

OPTIMIZATIONS | STEAKHOUSE FINANCE - AUDIT

ID	Title	Category	Severity	Status
SFA-16	Unnecessary Utils Contract Dependency	Code Optimization	Optimization	● Resolved
SFA-17	Duplicate Functions <code>setHeaderlessPreference</code> And <code>setRemoveHeaderFlag</code>	Code Optimization	Optimization	● Resolved

SFA-16 | Unnecessary Utils Contract Dependency

Category	Severity	Location	Status
Code Optimization	● Optimization	contracts/KitchenCreatorSimple.sol (base): 25	● Resolved

Description

It is unnecessary to store the utils contract address in contract KitchenCreatorSimple, as it has not been used anywhere in the contract.

Recommendation

Remove unused dependency.

Alleviation

[SteakHouse Finance, 10/28/2025]: The team heeded the advice and resolved the issue in commit [10ea30ea2864ac25379541e38355a631a0c5a1e8](#)

SFA-17 Duplicate Functions `setHeaderlessPreference` And `setRemoveHeaderFlag`

Category	Severity	Location	Status
Code Optimization	● Optimization	contracts/Kitchen.sol (base): 221~235; contracts/KitchenFactory.sol (base): 325~334	● Resolved

Description

The mentioned two functions serves the exact same purpose. There is no need to implement two of them.

Recommendation

Remove one of the functions.

Alleviation

[SteakHouse Finance, 10/28/2025]: The team heeded the advice and resolved the issue in commit [051f53671850a92d61e6811816354a866d878d86](#)

APPENDIX | STEAKHOUSE FINANCE - AUDIT

Audit Scope

steaktech/Smart_Contracts_Kitchen

-  contracts/KitchenBondingCurve.sol
-  contracts/KitchenDeployer.sol
-  contracts/KitchenFactory.sol
-  contracts/KitchenGraduation.sol
-  contracts/Kitchen.sol
-  contracts/KitchenCreatorBasicAdvanced.sol
-  contracts/KitchenCreatorSimple.sol
-  contracts/KitchenCurveMaths.sol
-  contracts/KitchenStorage.sol
-  contracts/KitchenUtils.sol
-  contracts/SteakLockers.sol
-  contracts/KitchenEvents.sol

Finding Categories

Categories	Description
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Categories	Description
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Code Optimization	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

SteakHouse Finance - Audit Security Assessment | CertiK Assessed on May 8th, 2026 | Copyright © CertiK

